



**SECRETARIA DE EDUCACIÓN PÚBLICA  
TECNOLÓGICO NACIONAL DE MÉXICO  
INSTITUTO TECNOLÓGICO DE VERACRUZ**

**ASIGNATURA:**

BD NOSQL.

**CARRERA:**

ING. SISTEMAS COMPUTACIONALES.

**HORA:**

03-04 P.M.

**ALUMNO:**

RAYON HERNANDEZ RAFAEL. - 22020821

**TEMA:**

CONEXIÓN CON MONGODB ATLAS Y MIGRACIÓN DE DATOS  
EN LOS DOS SENTIDOS (NUBE ↔ LOCAL).

**SEMESTRE:**

AGOSTO - DICIEMBRE 2026.

VERACRUZ, VER. A 24 DE SEPTIEMBRE DE 2026.



## Introducción

En esta práctica conecté mi base de datos local (la que tengo corriendo en un contenedor con Podman) con una base de datos en la nube (MongoDB Atlas, que está alojada en los servidores de Amazon AWS).

El objetivo fue aprender a **mover datos de un lado a otro**: primero bajé información de la nube a mi computadora, y luego subí información de mi computadora a la nube. Esto es lo que en el mundo real se llama una **migración de datos**, y es algo que se hace mucho cuando una empresa cambia de servidor o quiere tener respaldos en distintos lugares.

## Preparación de la nube con MongoDB Atlas

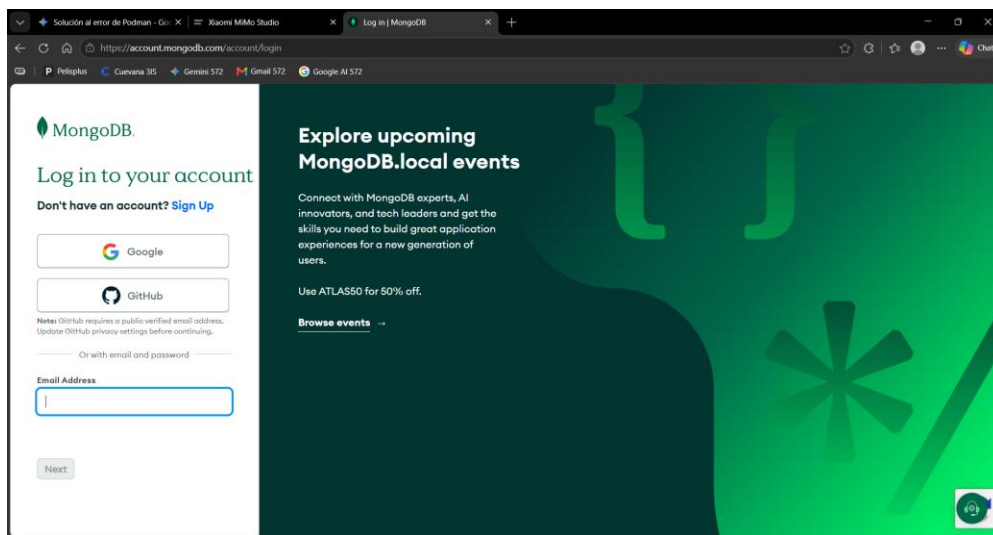
Antes de migrar nada, había que dejar lista la cuenta y el servidor en la nube.

### Iniciar sesión en Atlas desde la terminal

Atlas tiene su propia herramienta de línea de comandos (se llama **atlas**) con la que puedo administrar todo sin entrar a la página web. El primer paso fue "presentarme" con mi cuenta:

```
atlas auth login
```

Esto abre el navegador y me pide iniciar sesión con mi cuenta de MongoDB Atlas. Así la terminal queda autorizada para trabajar con mi clúster.



### Crear el clúster (mi servidor en la nube)

Un **clúster** es básicamente el "espacio en la nube" donde van a vivir mis bases de datos. Lo creé con este comando:

```
atlas clusters create MiClusterAWS --provider AWS --region us-east-1 --tier M0
```

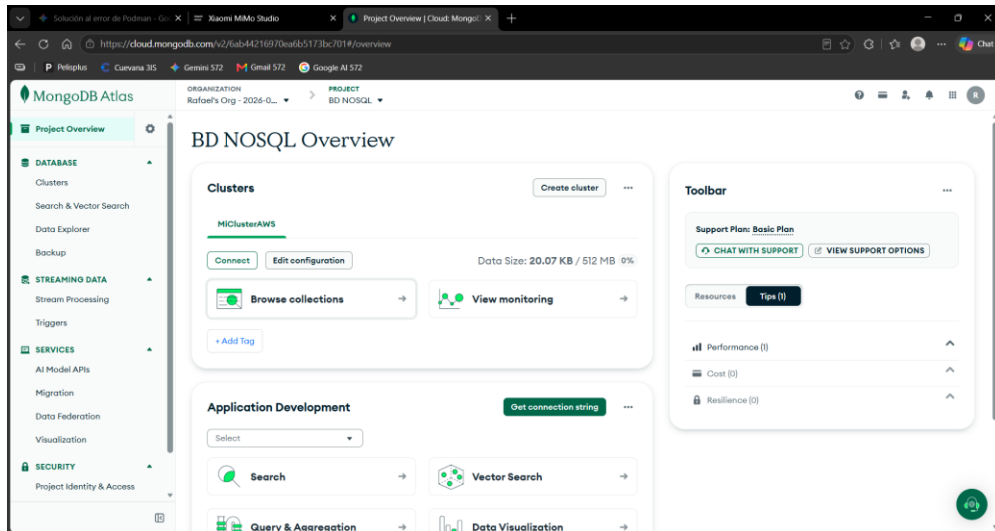
**¿Qué significa cada parte?**

**clusters create MiClusterAWS** → crear un clúster y ponerle de nombre "MiClusterAWS".

**--provider AWS** → que lo hospede Amazon Web Services.

**--region us-east-1** → en la región de Norteamérica (Virginia del Norte). Se elige la más cercana para que la conexión sea más rápida.

**--tier M0** → el plan gratuito (sandbox). Es el plan gratis que da MongoDB para aprendizaje y proyectos pequeños.



## Crear usuario y abrir el acceso

Para poder conectarme necesito dos cosas: un **usuario con contraseña** y **permiso para entrar desde mi IP** (por seguridad, Atlas no deja entrar a cualquiera).

Primero, el usuario de base de datos:

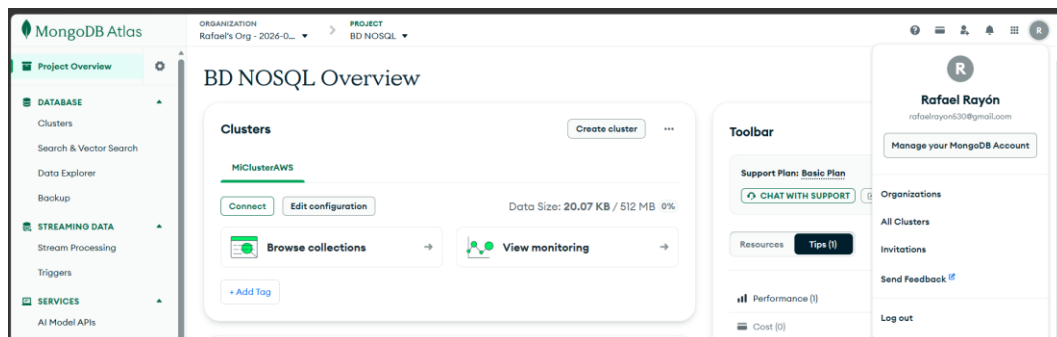
```
atlas dbusers create --username rafa --password TU_CONTRASEÑA --role readWriteAnyDatabase.
```

El rol **readWriteAnyDatabase** significa que este usuario puede leer y escribir en todas las bases de datos del clúster.

Segundo, autorizar mi dirección de IP:

```
atlas accessLists add --currentIp
```

El comando **--currentIp** detecta automáticamente la IP de mi computadora y la agrega a la lista de permitidos. Si no hago esto, Atlas rechaza la conexión, aunque la contraseña sea correcta.



## Preparar los dos entornos con datos de prueba

Para no mezclar las cosas, usé **dos bases de datos con nombres distintos**:

- **tienda\_podman** → la que vive en mi computadora (en el contenedor).
- **tienda\_atlas** → la que vive en la nube.

### Crear datos en mi computadora (local)

Entré a la consola de mi contenedor de Podman:

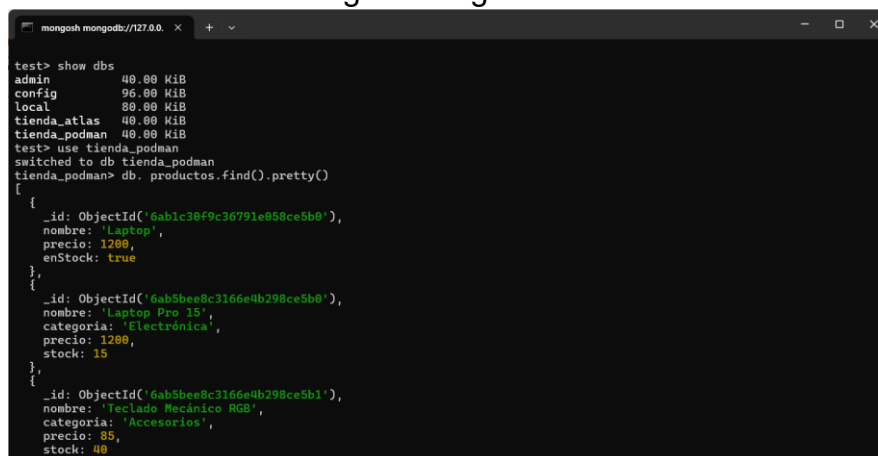
```
podman exec -it todoDB mongosh
```

Y dentro de la consola escribí esto para crear la base de datos local con tres productos de prueba:

```
use tienda_podman
```

```
db. productos. insertMany ([{nombre: "Laptop Gamer", categoría: "Cómputo", precio: 1500, origen: "Podman Local"}, {nombre "Ratón Inalámbrico", categoría: "Accesorios", precio: 25, origen: "Podman Local"}, {nombre: "Teclado Mecánico", categoría: "Accesorios", precio: 80, origen: "Podman Local"}])
```

El campo **origen** lo agregué yo a propósito, para poder distinguir fácilmente qué datos vienen de dónde cuando haga las migraciones.



```
test> show dbs
admin      40.00 KiB
config     96.00 KiB
local      80.00 KiB
tienda_atlas 40.00 KiB
tienda_podman 40.00 KiB
test> use tienda_podman
switched to db tienda_podman
tienda_podman> db. productos.find().pretty()
{
  _id: ObjectId('6ab1c30f9c36791e058ce5b0'),
  nombre: 'Laptop',
  precio: 1200,
  enStock: true
},
{
  _id: ObjectId('6ab5bee8c3166e4b298ce5b0'),
  nombre: 'Laptop Pro 15',
  categoría: 'Electrónica',
  precio: 1200,
  stock: 15
},
{
  _id: ObjectId('6ab5bee8c3166e4b298ce5b1'),
  nombre: 'Teclado Mecánico RGB',
  categoría: 'Accesorios',
  precio: 85,
  stock: 40
}
```

### Crear datos en la nube

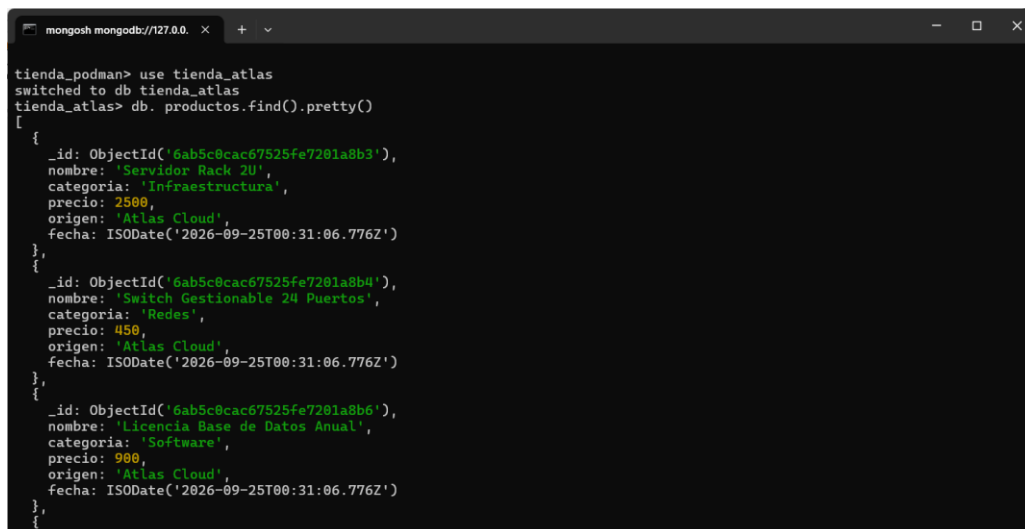
Me conecté directamente al clúster de Atlas desde la terminal:

```
Atlas clusters connect MiClusterAWS --username rafa.
```

Y dentro de esa consola creé la base de datos de la nube con cuatro registros de infraestructura:

```
use tienda_atlas
db.productos.insertMany([ { nombre: "Servidor Rack 2U", categoría:
"Infraestructura", precio: 2500, origen: "Atlas Cloud", fecha: new Date() }, {
nombre: "Switch Gestionable 24 Puertos", categoría: "Redes", precio: 450,
origen: "Atlas Cloud", fecha: new Date() }, { nombre: "Firewall Hardware
Enterprise", categoría: "Seguridad", precio: 1800, origen: "Atlas Cloud",
fecha: new Date() }, { nombre: "Licencia Base de Datos Anual", categoría:
"Software", precio: 900, origen: "Atlas Cloud", fecha: new Date() } ])
```

El campo **fecha** con **new Date ()** guarda automáticamente el día y la hora en que se insertó cada registro.



```
mongosh mongodb://127.0.0.1:27020
tienda_podman> use tienda_atlas
switched to db tienda_atlas
tienda_atlas> db.productos.find().pretty()
[
  {
    _id: ObjectId('6ab5c9cac67525fe7201a8b3'),
    nombre: 'Servidor Rack 2U',
    categoría: 'Infraestructura',
    precio: 2500,
    origen: 'Atlas Cloud',
    fecha: ISODate('2026-09-25T00:31:06.776Z')
  },
  {
    _id: ObjectId('6ab5c9cac67525fe7201a8b4'),
    nombre: 'Switch Gestionable 24 Puertos',
    categoría: 'Redes',
    precio: 450,
    origen: 'Atlas Cloud',
    fecha: ISODate('2026-09-25T00:31:06.776Z')
  },
  {
    _id: ObjectId('6ab5c9cac67525fe7201a8b6'),
    nombre: 'Licencia Base de Datos Anual',
    categoría: 'Software',
    precio: 900,
    origen: 'Atlas Cloud',
    fecha: ISODate('2026-09-25T00:31:06.776Z')
  },
  {
    _id: ObjectId('6ab5c9cac67525fe7201a8b5'),
    nombre: 'Firewall Hardware Enterprise',
    categoría: 'Seguridad',
    precio: 1800,
    origen: 'Atlas Cloud',
    fecha: ISODate('2026-09-25T00:31:06.776Z')
  }
]
```

## Migración 1: bajar datos de la nube a mi computadora

### Exportar los datos desde Atlas

Para sacar los datos de la nube usé **mongoexport**, que convierte los registros en un archivo de texto (formato JSON). Lo corrí **desde mi contenedor de Podman**, conectándose por internet al clúster de Atlas:

```
podman      exec      -it      todoDB      mongoexport      --
uri="mongodb+srv://rafa:TU_CONTRASEÑA@miclusteraws.wivpckn.mongo
db.net/tienda_atlas"      --collection=productos      --
out=/tmp/productos_desde_atlas.json --jsonArray
```

### Explicación rápida:

**podman exec -it todoDB ...** → ejecuta el comando adentro de mi contenedor.

**--uri="mongodb+srv://..."** → la dirección de conexión al clúster de Atlas (incluye usuario, contraseña, nombre del clúster y base de datos).

**--collection=productos** → qué colección quiero exportar.

**--out=/tmp/productos\_desde\_atlas.json** → nombre y ruta del archivo que se va a generar.

**--jsonArray** → guarda todo dentro de un solo arreglo JSON, que es más fácil de leer e importar después.

Resultado: se descargaron **4 registros** al archivo **/tmp/productos\_desde\_atlas.json**.

### Importar esos datos en una base local nueva

Con el archivo ya en mi máquina, lo cargué hacia el MongoDB local. Fíjate que mantengo el mismo nombre de base de datos (**tienda\_atlas**), pero ahora vivirá en mi compu:

```
podman      exec      -it      todoDB      mongoimport      --db=tienda_atlas      --
collection=productos --file=/tmp/productos_desde_atlas.json --jsonArray
```

**mongoimport** hace el proceso inverso a **mongoexport**: lee el archivo JSON y lo mete a la base de datos.

## Verificar que todo llegó bien

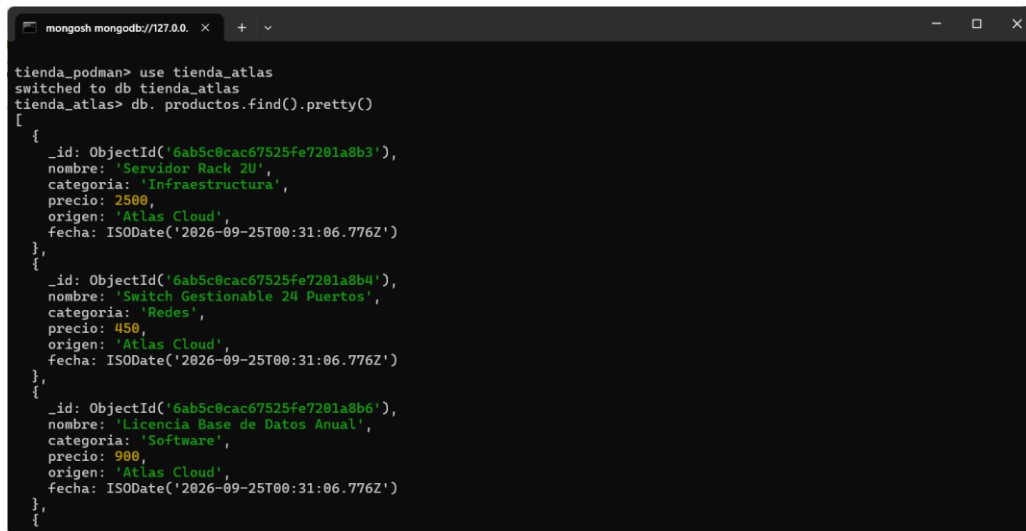
Entré de nuevo a la consola local:

```
podman exec -it todoDB mongosh
```

Y revisé que los datos estuvieran ahí:

```
use tienda_atlas show collections db. productos. find (). pretty ()
```

El `pretty ()` solo sirve para que los documentos se vean ordenados en pantalla, en vez de una línea larguísima.



```

tienda_podman> use tienda_atlas
switched to db tienda_atlas
tienda_atlas> db. productos.find().pretty()
[
  {
    _id: ObjectId('6ab5c0cac67525fe7201a8b3'),
    nombre: 'Servidor Rack 2U',
    categoria: 'Infraestructura',
    precio: 2500,
    origen: 'Atlas Cloud',
    fecha: ISODate('2026-09-25T00:31:06.776Z')
  },
  {
    _id: ObjectId('6ab5c0cac67525fe7201a8b4'),
    nombre: 'Switch Gestionable 24 Puertos',
    categoria: 'Redes',
    precio: 450,
    origen: 'Atlas Cloud',
    fecha: ISODate('2026-09-25T00:31:06.776Z')
  },
  {
    _id: ObjectId('6ab5c0cac67525fe7201a8b6'),
    nombre: 'Licencia Base de Datos Anual',
    categoria: 'Software',
    precio: 900,
    origen: 'Atlas Cloud',
    fecha: ISODate('2026-09-25T00:31:06.776Z')
  }
]

```

## Migración 2: subir datos de mi computadora a la nube

### Exportar los datos locales a un archivo

Ahora al revés. Primero genero el archivo JSON con los datos que creé en mi computadora:

```
podman exec -it todoDB mongoexport --db=tienda_podman --collection=productos --out=/tmp/tienda_podman_para_atlas.json --jsonArray
```

En este caso no necesito la URI, porque los datos están justo ahí en el contenedor.

### Importar el archivo directo a Atlas

Con el mismo contenedor (que tiene salida a internet) subí el archivo al clúster:

```

podman          exec          -it          todoDB          mongoimport          --
uri="mongodb+srv://rafa:TU_CONTRASEÑA@miclusteraws.wivpckn.mongod
b.net/tienda_podman"          --collection=productos          --
file=/tmp/tienda_podman_para_atlas.json --jsonArray

```

Fíjate que en la URI pongo **tienda\_podman**: así se crea esa base de datos dentro del clúster de Atlas, y ahí se quedan mis datos locales.

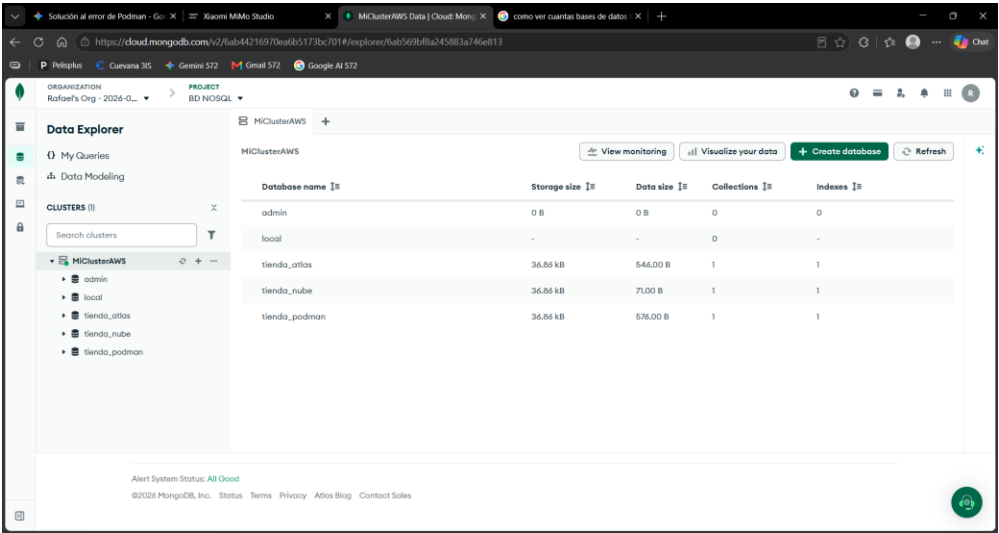
Verificación final en la nube

Me conecté otra vez al clúster:

```
Atlas clusters connect MiClusterAWS --username rafa.
```

Y comprobé que ahora tengo las dos bases de datos conviviendo en la nube:

```
show dbs use tienda_podman db. productos. find (). pretty ()
```



Resumen de cómo quedó todo

| Dónde está | Servidor           | Base de datos | Colección | De dónde salieron esos datos                      |
|------------|--------------------|---------------|-----------|---------------------------------------------------|
| Local      | Podman (todoDB)    | tienda_podman | productos | Los creé directo en mi compu (3 productos)        |
| Local      | Podman (todoDB)    | tienda_atlas  | productos | Los bajé desde la nube (4 productos)              |
| Nube       | AWS (MiClusterAWS) | tienda_atlas  | productos | Se crearon originalmente en la nube (4 productos) |
| Nube       | AWS (MiClusterAWS) | tienda_podman | productos | Los subí desde mi compu (3 productos)             |

## Conclusión

Con esta actividad logré tener mis datos funcionando en dos lugares al mismo tiempo y aprendí a moverlos de un lado a otro cuando yo quiera. La verdad es que no fue tan complicado: solo se necesitan dos herramientas, **mongoexport** para sacar los datos en un archivo JSON y **mongoimport** para volverlos a meter.

Lo que más me llamó la atención es que el archivo JSON funciona como un "puente" entre el mundo local y la nube: no importa si los datos están en mi contenedor de Podman o en un servidor de Amazon, mientras pueda exportarlos e importarlos, puedo pasarlos de un lado a otro.

Esto es exactamente lo que hace una empresa cuando quiere cambiar de proveedor de nube o tener respaldos de seguridad.

También aprendí que en Atlas la seguridad importa mucho: sin crear un usuario con contraseña y sin agregar mi IP a la lista de acceso, la conexión simplemente no funciona, aunque los demás datos estén bien.